



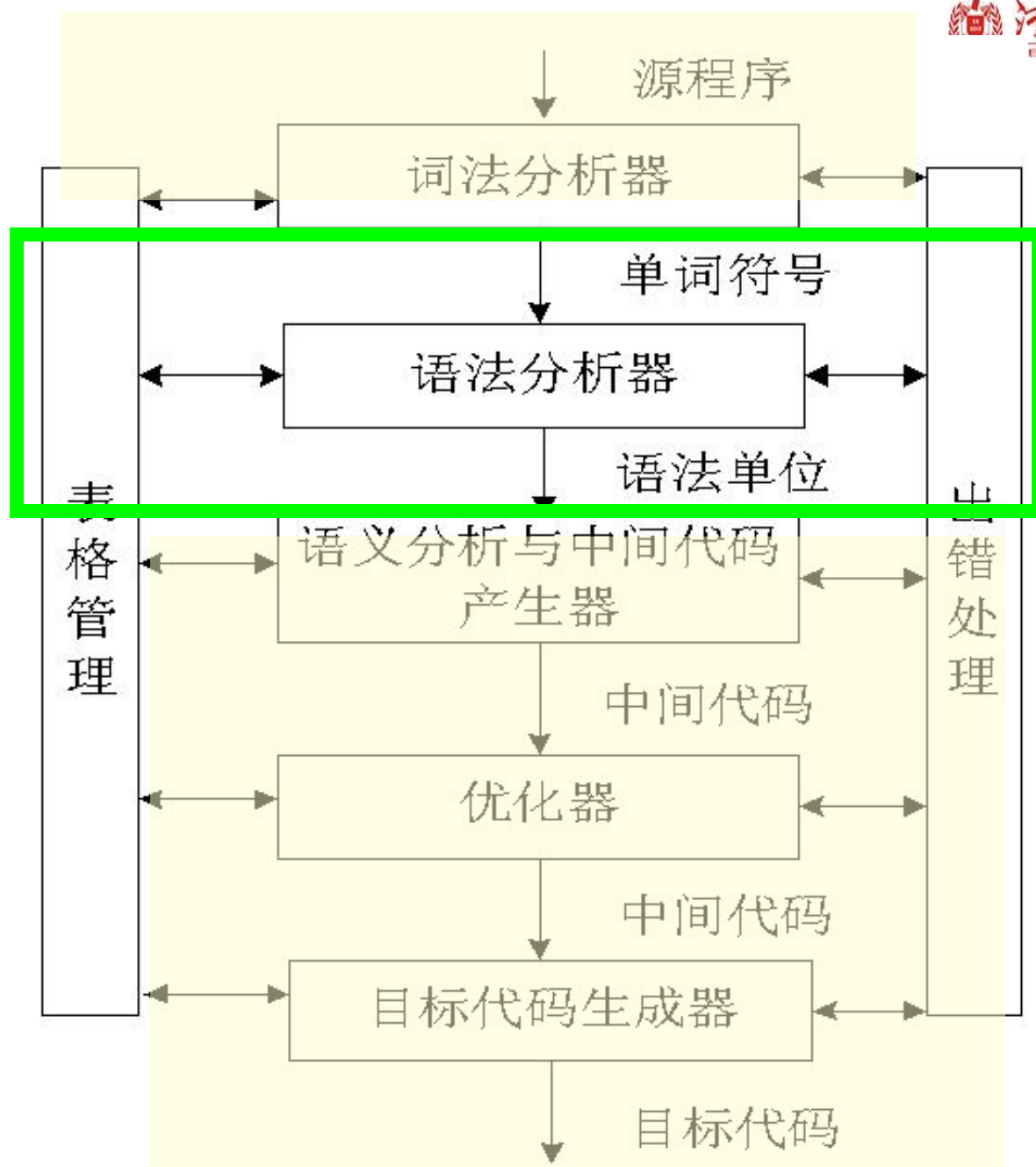
第三章 语法分析



语法分析是编译过程的核心。

语法分析是对高级程序设计语言的句子结构进行分析，它的任务是识别输入的单词序列是否符合语言的语法规则，如果符合通常就生成语法树。

根据生成语法树的方法，可将语法分析分成自上而下方法和自下而上方法。



回顾

一个文法 G 抽象地表示为四元组

$G =$

(V_n, V_t, P, S)

其中 V_n 表示非终结符号

V_t 表示终结符号, $V_n \cup V_t$ (字母表),

$V_n \cap V_t = \phi$

S 是开始符号,

P 是产生式, 形如: $\alpha \rightarrow \beta$ ($\alpha \in V^+$ 且至少含有一个非终结符号, $\beta \in V^*$)



上下文无关文法

$E \rightarrow E + E | E * E | (E) | i$

回顾



- **推导**：连续使用产生式右部去替换左部某个非终结符的过程,得到的连续序列称为一个推导。
- **最左推导**：在整个推导过程中,任何一步推导 $\alpha \Rightarrow \beta$ 都是对 α 中最左边的非终结符进行替换。
- **最右推导**：在整个推导过程中,任何一步推导 $\alpha \Rightarrow \beta$ 都是对 α 中最右边的非终结符进行替换。

回顾

句型：若 S 是文法 G 的开始符号，从开始符号 S 出发推导出的符号串称为文法 G 的一个句型。

句子：若 x 是文法 G 的一个句型，且 $x \in V_t^*$ 即 x 仅由终结符号组成，则称 x 为 $G(S)$ 的句子

文法 G 所产生的语言定义为：

$L(G) = \{x \mid S \Rightarrow x, \text{ 其中 } S \text{ 为文法的开始符号, } x \in V_t^*\}$ 。即：一个文法 G 可以推导出的所有句子构成的集合，称为 G 产生的语言。

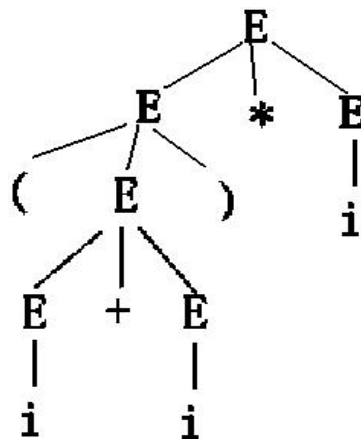


3.2.4 语法树

语法树：用一棵树来表示句型的推导称为语法树，也可称为分析树，通常表示为一棵倒立的树。

- 每个结点都有一个标记，该标记属字母集中的一个符号，根由开始符号标记。

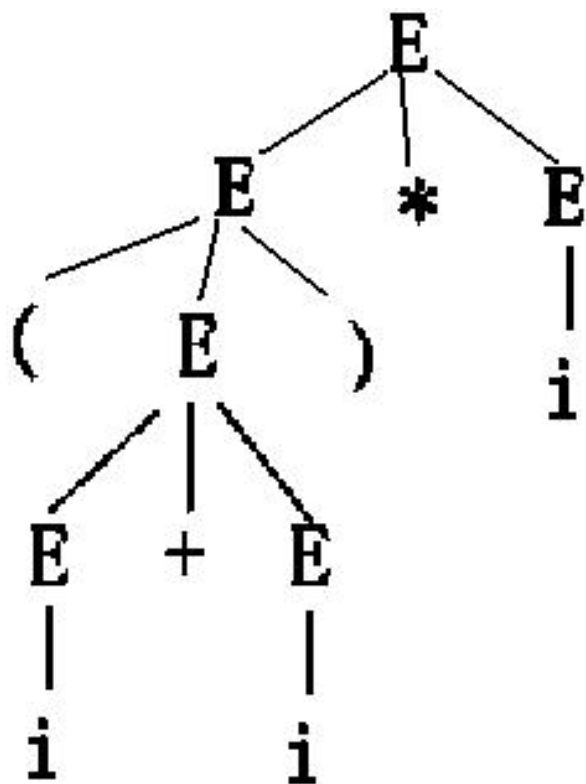
- 当某个非终结符被它的某个候选式所替换时，就产生相应的下一层的结点，候选式中自左至右的每个符号对应一个新的结点，并标记它，画出其与父结点之间的连线。



例：对文法 $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P: E \rightarrow E + E \mid E * E \mid (E) \mid i$

句子 $(i+i)*i$ 的语法树：



若 S 是文法 G 的开始符号，从开始符号 S 出发推导出的符号串称为文法 G 的一个句型。

在语法树表示的推导过程中的任何时刻，没有后代的端末结点自左至右排列起来就是一个句型

一棵语法树表示了一个句型很多可能的不同推导过程。(包括最左推导和最右推导)

文法: $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P: E \rightarrow E + E \mid E * E \mid (E) \mid i$

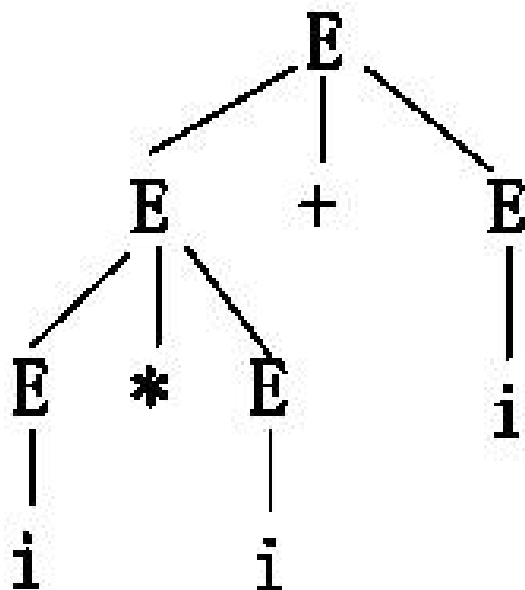
推导: $E \Rightarrow E + E \Rightarrow E * E + E \Rightarrow i * E + E \Rightarrow i * i + E \Rightarrow$
 $i * i + i$

$E \Rightarrow E + E \Rightarrow E + i \Rightarrow E * E + i \Rightarrow E * i + i \Rightarrow$

句型: $i * i + i$
 $E \quad E + E \quad E * E + E \quad i * E + E \quad i * i + E \quad i * i + i$

句子: $i * i + i$

语法树



句子: 若 x 是文法 G 的一个句型, 且
 $x \in V_t^*$ 即 x 仅由终结
符号组成, 则称 x 为
 $G(S)$ 的句子

◆ 从一个句型到另一个句型(句子)的推导过程不是唯一的。(语法树可以相同)

一棵语法树表示了一个句型很多可能的不同推导过程。

◆ 对给定的文法，定义的语言是由利用所有的产生式经过各种方式推导出所有可能的句子构成的，并没有规定推导使用产生式的顺序。

思考：一个推导过程对应唯一的一棵语法树，一棵语法树对应多个推导过程。对给定句子的推导过程是不是唯一地对应一棵语法树？

3.2.5 二义文法

文法: $G = (\{E\}, \{i, +, *, (,)\}, P, E)$

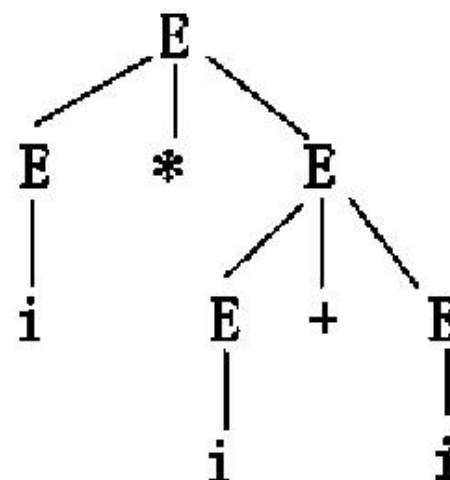
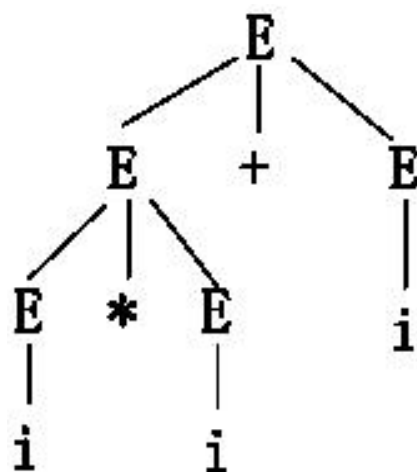
$P: E \rightarrow E + E \mid E * E \mid (E) \mid i$

推导:

(1) $E \Rightarrow E + E \Rightarrow E * E + E \Rightarrow i * E + E \Rightarrow i * i + E \Rightarrow i * i + i$

(2) $E \Rightarrow E * E \Rightarrow i * E \Rightarrow i * E + E \Rightarrow i * i + E \Rightarrow i * i + i$

语法树



3.2.5 二义文法

定义：如果一个文法存在某个句型对应两棵不同的语法树，则说这个文法是**二义**的

- 对二义文法中的某个句子的分析不是唯一的，增加了算法的复杂性。因此有时希望文法是无二义的。
- 但是二义文法有时也是有用的。



3.2.6 消除二义性

证明下述文法是二义文法。

$$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$$
$$E \rightarrow e$$

句子 $\text{if } E \text{ then if } E \text{ then } S \text{ else } S$



只有当产生一个语言的所有文法都是二义的，称该语言为二义的。

3.2.7 程序设计语言文法描述

程序设计语言的定义

语法

定义程序的一组形式规则，用它可以形成和产生一个形式上正确的程序。

语义

是一组规则的集合，用以定义语法正确的单词符号和语法单位的含义。

语用

是有关程序设计技术和语言成分的使用方法。



1 表达式

表达式是构成程序设计语言中语句的最基本的成分。

- 表达式由运算对象和运算符组成。
- 分为逻辑表达式、关系表达式和算术表达式
- 运算符之间的优先关系和结合性规定了表达式的计算次序。



算术表达式

$\langle \text{算术表达式} \rangle \rightarrow \langle \text{常量} \rangle | \langle \text{变量} \rangle | (\langle \text{算术表达式} \rangle)$
 $| \langle \text{算术表达式} \rangle \langle \text{算术运算符} \rangle \langle \text{算术表达式} \rangle$

$\langle \text{算术表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle + \langle \text{项} \rangle$
 $| \langle \text{算术表达式} \rangle - \langle \text{项} \rangle | \langle \text{项} \rangle$

$\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle * \langle \text{因子} \rangle | \langle \text{项} \rangle / \langle \text{因子} \rangle | \langle \text{因子} \rangle$

$\langle \text{因子} \rangle \rightarrow (\langle \text{算术表达式} \rangle) | \langle \text{常量} \rangle | \langle \text{变量} \rangle$

$\langle \text{变量} \rangle \rightarrow \langle \text{标识符} \rangle$

$\langle \text{常量} \rangle \rightarrow \langle \text{整型常量} \rangle | \langle \text{实型常量} \rangle$

逻辑表达式

<逻辑表达式> → <布尔常量>

<逻辑表达式> → <布尔变量>

<逻辑表达式> → (<关系表达式>)

<逻辑表达式> → not <逻辑表达式>

<逻辑表达式> → <逻辑表达式> and <逻辑表达式>

<逻辑表达式> → <逻辑表达式> or <逻辑表达式>



关系表达式

$\langle \text{关系表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle \langle \text{关系运算符} \rangle \langle \text{算术表达式} \rangle$

$\langle \text{关系运算符} \rangle \rightarrow <|>|<>|<=|>=|==$





2 语句

程序由各种语句构成。

- 语句
 - 说明性语句
 - 可执行语句
- 说明性语句旨在定义各种不同数据类型的变量或运算，不需要由编译程序生成目标代码，主要用来告诉编译程序一些实体的属性，供编译程序生成目标代码时使用。
- 可执行语句旨在描述程序的动作，需要由编译程序生成目标代码来实现它的语义。

说明语句

<说明语句> → <常量说明>|<变量说明>

<常量说明> → const <标识符> = <常数>

<变量说明> → var <变量表>:<类型>

<变量表> → <变量>|<变量表>,<变量>

<变量> → <标识符>

<类型> → integer|real|char|boolean



执行语句

$\langle \text{执行语句} \rangle \rightarrow \langle \text{读语句} \rangle | \langle \text{写语句} \rangle | \langle \text{赋值语句} \rangle | \langle \text{控制语句} \rangle | \langle \text{复合语句} \rangle$

$\langle \text{读语句} \rangle \rightarrow \text{read} (\langle \text{变量} \rangle)$

$\langle \text{写语句} \rangle \rightarrow \text{write} (\langle \text{变量} \rangle)$

$\langle \text{赋值语句} \rangle \rightarrow \langle \text{变量} \rangle = \langle \text{表达式} \rangle$

$\langle \text{表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle | \langle \text{逻辑表达式} \rangle | \langle \text{关系表达式} \rangle$

$\langle \text{控制语句} \rangle \rightarrow \langle \text{if语句} \rangle | \langle \text{while语句} \rangle | \langle \text{for语句} \rangle | \langle \text{dowhile语句} \rangle$

$\langle \text{复合语句} \rangle \rightarrow \text{begin} \langle \text{语句表} \rangle \text{end}$

.....

3 程序

程序是语言中最大的语法单位。设计时应考虑程序的总体结构，各子程序之间的关系，如何构成一个程序等。

$\langle \text{程序} \rangle \rightarrow \text{program } \langle \text{标识符} \rangle;$

$\langle \text{程序体} \rangle$

$\langle \text{程序体} \rangle \rightarrow \langle \text{常量说明} \rangle \langle \text{变量说明} \rangle \langle \text{复合句} \rangle.$



符合SAMPLE语言规范的源程序

```
/*this is a sample program writing in Sample language*/  
program example1;  
/*used for illustrating compiling process*/  
var  
    a,b,c:integer;  
    x:char;  
begin  
    a=2;  
    read(b);  
    read(c);  
    x=' stu' ;  
    if a>0 and (x<>' ' ) then b=a+b*c;  
    else  
        for b= 1 to 100 do c=c+b;  
        while c>=1 do  
            begin  
                b=b*c;  
                c=c-1;  
            end;  
        do a=10; while a>b;  
    end.
```

例：判定输入串 $(i+i)*i$ 是否是下述文法的句子？

$G = (\{E\}, \{i, +, *, (,)\}, P, E)$

$P:$ (1) $E \rightarrow E + E$

(2) $E \rightarrow E * E$

(3) $E \rightarrow (E)$

(4) $E \rightarrow i$

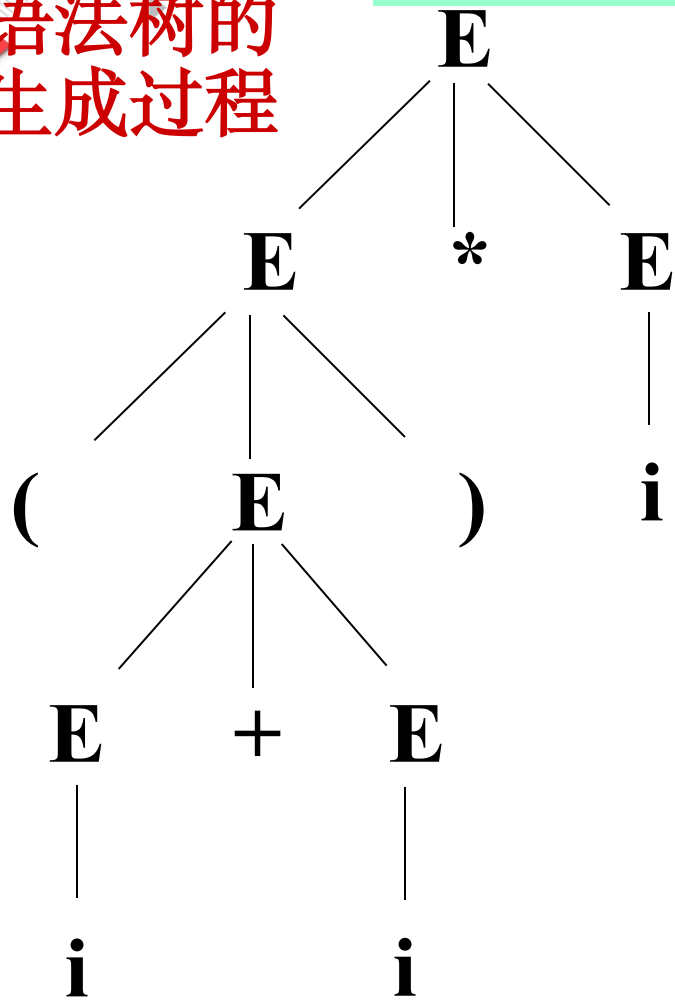
使用最左推导：

$$\begin{aligned} E &\Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (i + E) * E \\ &\Rightarrow (i + i) * E \Rightarrow (i + i) * i \end{aligned}$$

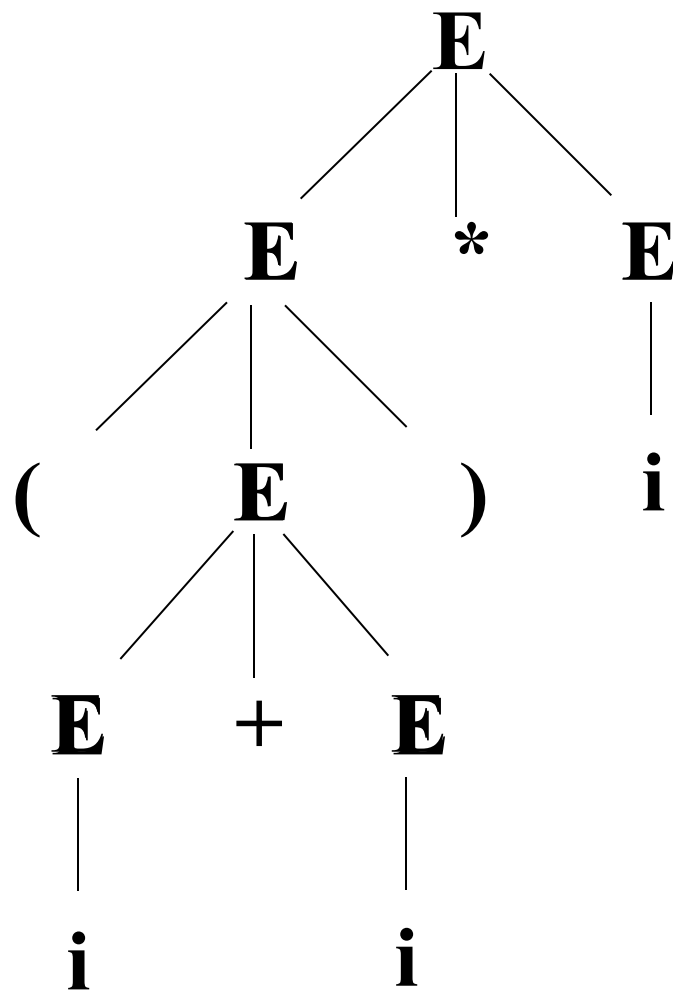
能够从开始符号出发**推导**出给定的输入串，因此是句子

$E \Rightarrow E * E \Rightarrow (E) * E \Rightarrow (E + E) * E \Rightarrow (i + E) * E$
 $\Rightarrow (i + i) * E \Rightarrow (i + i) * i$

语法树的
生成过程



自上而下建立



自下而上建立

3.3 自上而下的语法分析

➤ 自上而下语法分析

- 从推导的角度看，从开始符号出发，使用最左推导，试图推导出与输入符号串相同的句子。
- 从语法树的角度看，从根节点出发，反复使用文法所有可能的产生式，自上而下地为输入符号串建立一棵语法树。



例1：对于文法G1[S]:

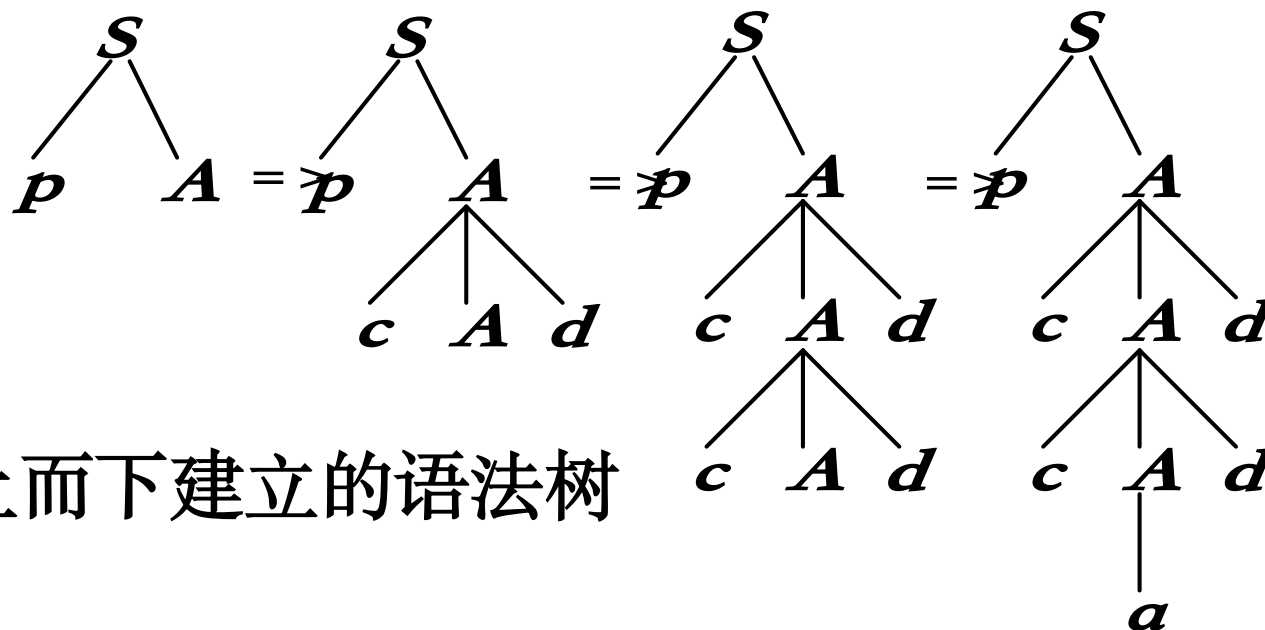
$$S \rightarrow pA$$

$$S \rightarrow qB$$

$$A \rightarrow cAd$$

$$A \rightarrow a$$

对输入串pccadd，最左推导过程是：

$$S \Rightarrow pA \Rightarrow pcAd \Rightarrow pccAdd \Rightarrow pccadd$$


自上而下建立的语法树

3.3.1 自上而下分析方法中的问题探究

1 确定的自上而下分析面临的问题

从文法的开始符号出发，根据当前的输入符号**唯一的确定选用哪个产生式**替换相应非终结符往下推导，或构造一棵相应的语法树的过程。



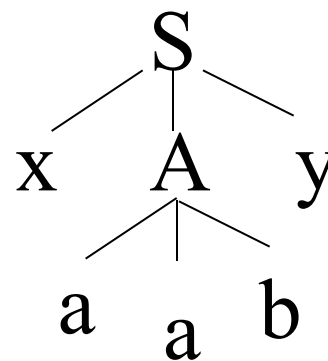
例2：设有文法

$$(1) \quad S \rightarrow xAy$$

$$(2) \quad A \rightarrow ab$$

$$(3) \quad A \rightarrow a$$

输入串 **xay** 的分析过程

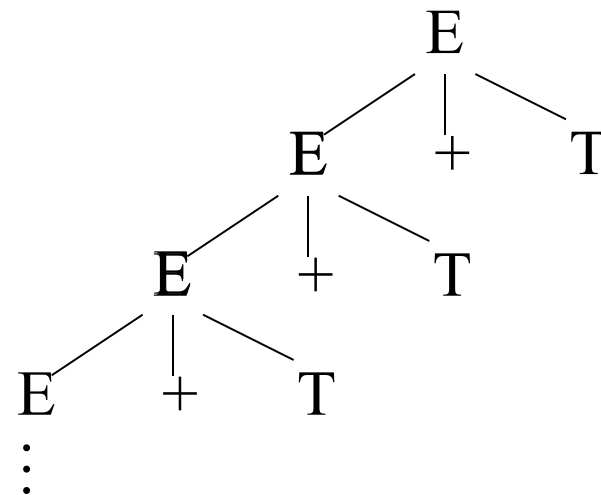




现有输入串 $i*i+i$ ，分析过程是：

$$(2) \quad T \rightarrow T * F \mid F$$

$$(3) \quad F \rightarrow (E) \mid i$$



当某个非终结符有多个产生式候选时，可能带来如下问题：

1. **回溯** 存在形如 $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ 的产生式，即某非终结符存在多个候选式的前缀相同（称为公共左因子），当一个非终结符用某一个候选匹配成功时，这种匹配可能是暂时的。使得分析过程中可能需要进行大量回溯。



2. **无限循环** 文法中存在形如 $A \rightarrow A\alpha$ 的产生式，称文法为左递归的，分析过程又使用最左推导，就会使分析陷入无限循环。

问题的解决方法

- 消除回溯

- 消除左递归

改造为等价文法



2 消除回溯

方法是：反复“提取公共左因子”，使得文法的每个非终结符号的各个候选式的首终结符集两两不相交，来避免回溯。

$$A \rightarrow \alpha\beta_1 | \alpha\beta_2$$

替换为： $A \rightarrow \alpha A'$

$$A' \rightarrow \beta_1 | \beta_2$$

设产生式为： $A \rightarrow \delta\beta_1 | \delta\beta_2 | \dots | \delta\beta_n | \gamma_1 | \gamma_2 | \dots | \gamma_m$

替换为：

$$A \rightarrow \delta A' | \gamma_1 | \gamma_2 | \dots | \gamma_m$$

$$A' \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$$

• 例4：有如下两个产生式：

$\langle \text{IF语句} \rangle \rightarrow \text{if } E \text{ then } S1 \text{ else } S2$

$\langle \text{IF语句} \rangle \rightarrow \text{if } E \text{ then } S1$

提取左因子后：

$\langle \text{IF语句} \rangle \rightarrow \text{if } E \text{ then } S1 \ B$

$B \rightarrow \text{else } S2 \mid \epsilon$

练习

$$(1) S \rightarrow xAy$$

$$(2) A \rightarrow ab$$

$$(3) A \rightarrow a$$

提取下述文法的左因子

$$(1) S \rightarrow xAy$$

$$(2) A \rightarrow aB$$

$$(3) B \rightarrow b|\varepsilon$$

3 消除左递归

- 若文法G中存在某个非终结符A, 存在推导 $A \xRightarrow{+} A\alpha$, 则称文法是**左递归**的。
- 若文法G中有形如 $A \rightarrow A\alpha$ 的产生式, 则称文法是**直接左递归**的。
- 若文法G中没有形如 $A \rightarrow A\alpha$ 的产生式, 但是A经过有限步推导可以得到 $A \xRightarrow{+} A\alpha$, 则称文法是**间接左递归**的。

假定产生式为： $A \rightarrow A\alpha \mid \beta$ ，
将其替换为形式等价的产生式组：

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \varepsilon \end{aligned}$$

例 5 文法G(E):

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid i$$

经消去直接左递归后变成：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

① 直接左递归的消除：

一般而言，若产生式形式为：

$$A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_m | \beta_1 | \beta_2 | \dots | \beta_n$$

其中 α_i ($i=1 \dots m$) 都不为 ε ， β_j ($j=1 \dots n$) 均不以 A 开头。可以把 A 的产生式改写为：


$$\begin{aligned} A &\rightarrow \beta_1 A' | \beta_2 A' | \dots | \beta_n A' \\ A' &\rightarrow \alpha_1 A' | \alpha_2 A' | \dots | \alpha_m A' | \varepsilon \end{aligned}$$



练习

消去文法G[S]的左递归

$$S \rightarrow (T) \mid a + S \mid a$$

$$T \rightarrow T, S \mid S$$

答案：

$$S \rightarrow (T) \mid a + S \mid a$$

$$T \rightarrow ST'$$

$$T' \rightarrow ,ST' \mid \epsilon$$

②间接左递归的消除方法

例文法 $G(S)$:

$$S \rightarrow Qc | c$$

$$Q \rightarrow Rb | b$$

$$R \rightarrow Sa | a$$

虽没有直接左递归，但 S 、 Q 、 R 都是左递归的

$$S \Rightarrow Qc \Rightarrow Rbc \Rightarrow Sabc$$

一个间接左递归的消除方法:

☞ 采用先带入再消除直接左递归的方法

例 考虑文法 $G(S)$

$$S \rightarrow Qc | c$$

$$Q \rightarrow Rb | b$$

$$R \rightarrow Sa | a$$

- 令它的非终结符的排序为 R 、 Q 、 S 。
- 对于 R ，不存在直接左递归。
- 把 R 代入到 Q 的有关候选后，把 Q 的规则变为

$$Q \rightarrow Sab \mid ab \mid b$$

现在的Q不含直接左递归，把它代入到S的有关候选后，S变成

$$S \rightarrow Sabc \mid abc \mid bc \mid c$$

消除S的直接左递归后：

$$S \rightarrow abcS' \mid bcS' \mid cS'$$

$$S' \rightarrow abcS' \mid \varepsilon$$

$$Q \rightarrow Sab \mid ab \mid b$$

$$R \rightarrow Sa \mid a$$

关于Q和R的规则已是多余的，化简为：

$$S \rightarrow abcS' \mid bcS' \mid cS'$$

$$S' \rightarrow abcS' \mid \varepsilon$$

小结

1 语法树

2 二义文法

3 确定的自上而下的分析

公共左因子

左递归



问题：是否文法的每个非终结符的多个候选式不存在公共左因子，文法也不含左递归，就可以进行确定的自上而下的语法分析呢？